

类 Rust 词法分析与语法分析器设计报告

1. 课题概述

本课题要求依据《【Rust版】大作业1：词法和语法分析工具设计与实现 v5.0》给出的类 Rust 词法规则与语法规则，实现一个能够对输入源码进行词法分析和语法分析的前端工具。系统最终需要完成以下工作：

1. 读取类 Rust 源程序。
2. 将源程序切分为关键字、标识符、整数、运算符、界符等记号。
3. 在词法分析正确的基础上，按照给定文法检查程序结构是否合法。
4. 输出词法分析结果以及语法分析结果。

本实验采用 Rust 语言实现，系统被划分为两个相互协作的工程：

1. `My_Lexer`：完成词法分析。
2. `My_Parser`：复用 `My_Lexer` 的 token 序列，完成语法分析。

2. 需求分析

根据 PPT 的要求，系统至少需要支持以下内容。

2.1 词法规则

词法分析器需识别以下记号类型：

1. 关键字：`i32`、`let`、`if`、`else`、`while`、`return`、`mut`、`fn`、`for`、`in`、`loop`、`break`、`continue`
2. 标识符：`(字母|_)(字母|数字|_)*`，且不能与关键字相同
3. 数值：十进制整数
4. 运算符与特殊符号：`=`、`+`、`-`、`*`、`/`、`==`、`>`、`>=`、`<`、`<=`、`!=`、`&`、`->`、`..`、`...`
5. 界符与分隔符：`()` `{}` `[]` `;` `:` `,`
6. 注释：`//` 单行注释、`/* */` 块注释
7. 结束符：`#`

PPT 还特别强调两条识别要求：

1. `if123` 必须识别为标识符。
2. `if=123` 必须识别为三个单词：`if`、`=`、`123`。

2.2 语法规则

PPT 强制要求至少完成以下语法规则：

1. 0.1 变量属性
2. 0.2 类型
3. 0.3 左值
4. 1.1 基础程序
5. 1.2 语句
6. 1.3 返回语句

- 7. 1.4 函数输入
- 8. 1.5 函数输出
- 9. 2.0 变量声明
- 10. 2.1 变量声明语句
- 11. 2.2 赋值语句
- 12. 3.1 基本表达式
- 13. 3.2 比较运算
- 14. 3.3 加减运算
- 15. 3.4 乘除运算
- 16. 3.5 函数调用
- 17. 4.1 选择结构
- 18. 5.0 循环语句
- 19. 5.1 while 循环

在实际实现中，系统在满足最低要求的基础上，进一步扩展支持了 `else`、`else if`、`for`、`loop`、`break`、`continue`、引用、数组、元组、块表达式等结构。

3. 词法分析原理

3.1 基本思想

词法分析的任务是将字符流转换为 `token` 流。其核心思想是从左到右扫描输入串，根据当前字符和后续字符判断应当归约成哪一类记号。

本系统采用“手写扫描器”的实现方式，而非自动生成器。原因如下：

- 1. 本实验的词法规则规模较小，手写状态机更直接。
- 2. 手写扫描器便于控制保留字判定、最长匹配和错误恢复逻辑。
- 3. Rust 的模式匹配和枚举类型适合表达词法状态与 `token` 类型。

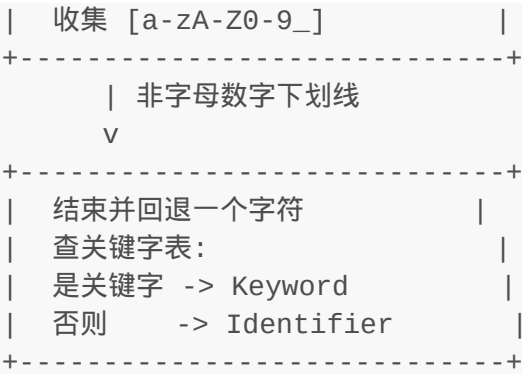
词法分析过程遵循以下原则：

- 1. 跳过空白符和注释。
- 2. 优先识别多字符运算符，保证最长匹配原则，例如优先识别 `>=` 而不是 `>` 加 `=`。
- 3. 识别字母或下划线开头的串，并在识别结束后判断其是关键字还是标识符。
- 4. 识别数字串作为整数。
- 5. 对无法识别的字符报告词法错误，并尽量继续向后扫描。

3.2 词法分析状态转换图

3.2.1 标识符与关键字识别





该状态机保证了 `if123` 会完整进入 `ID` 状态，最终识别为标识符，而不会被拆成关键字 `if` 与整数 `123`。

3.2.2 无符号整数识别



3.2.3 运算符与注释识别

```

Start
|
| '/'
v
SlashSeen
|-- '/' --> LineComment ----> 遇到 '\n' 或 EOF 结束，视为空白
|-- '*' --> BlockComment ----> 遇到 '*/' 结束，视为空白
|-- other --> 输出 Slash

Start
|
| '=' '>' '<' '!' '-' '.'
v
CheckNext
|-- 若与下一字符组成合法双字符记号
```

```
|      -> 输出 == >= <= != -> ..  
|  
|-- 否则  
      -> 输出单字符记号 = > < - .
```

该状态机体现了最长匹配原则。例如扫描到 `.` 时，若下一个字符仍然是 `.`，则输出 `..`，否则输出单独的 `.`。

3.3 词法分析器的关键实现点

3.3.1 保留字与标识符区分

扫描器先按统一规则识别“标识符候选串”，随后再查关键字表。这样可以保证：

1. `if` 被识别为关键字；
2. `if123` 被识别为标识符；
3. `while_loop` 被识别为标识符。

3.3.2 注释处理

系统支持两类注释：

1. `//` 单行注释：读到换行为止；
2. `/* */` 块注释：读到第一个匹配的 `*/` 为止。

注释在词法层面被视为空白，不会生成 token。

3.3.3 错误恢复

若遇到无法识别的字符，词法分析器记录错误位置与错误信息，并继续处理后续输入。这样可以在一次运行中输出更多错误信息，增强调试效率。

4. 语法分析原理

4.1 基本思想

语法分析的任务是判定 token 序列是否符合给定上下文无关文法，并进一步组织出程序的层次结构，如“函数声明”“语句块”“表达式”“控制流语句”等。

本系统采用递归下降分析方法实现语法分析器。递归下降分析的基本思想是：

1. 为每个主要非终结符编写一个分析函数。
2. 分析函数按照产生式右部的顺序依次匹配 token。
3. 通过函数间相互调用反映文法中的嵌套结构。
4. 在表达式部分按优先级分层处理，解决二义性与结合性问题。

之所以选用递归下降，主要原因如下：

1. 本实验的文法规模适中，手写递归下降易于实现与调试。
2. 程序结构与文法规则一一对应，可读性较高。
3. 对课程实验而言，递归下降更便于展示“从文法到程序”的设计过程。

4.2 语法分析总体流程

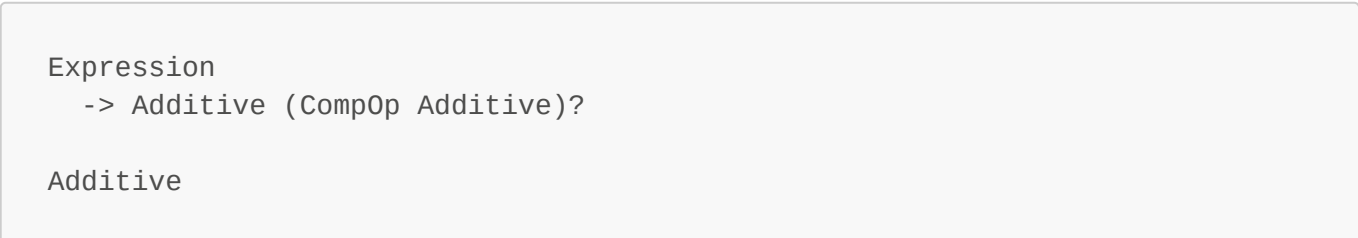


4.3 表达式优先级设计

为了正确处理 $a+b*c$ 、 $a>b+1$ 等表达式，本系统将表达式划分为多层：

- 1. 比较表达式层
- 2. 加减表达式层
- 3. 乘除表达式层
- 4. 一元表达式层
- 5. 后缀表达式层
- 6. 基本表达式层

对应关系如下：



```
-> Multiplicative { ('+'|'-') Multiplicative }

Multiplicative
-> Unary { ('*'|'/') Unary }

Unary
-> ('&' ['mut'] | '*') Unary
| Postfix

Postfix
-> Primary { Call | Index | Field }

Primary
-> NUM | ID | '(' Expression ')' | Block | IfExpr | LoopExpr | Array |
Tuple
```

这样可保证：

- 1. `*`、`/` 的优先级高于 `+`、`-`；
- 2. 比较运算优先级低于算术运算；
- 3. 函数调用、数组下标、元组字段访问等后缀结构优先于普通二元运算。

4.4 语法分析中的错误检测

语法错误通常出现在以下情况：

- 1. 缺少分隔符，如 `;`、`)`、`}`；
- 2. 结构不完整，如 `fn demo(` 后缺少参数列表闭合；
- 3. 语句位置错误，如 `return` 后面缺少合法表达式或分号；
- 4. 非法左值，如 `1=2;`。

系统在报告错误时会指出：

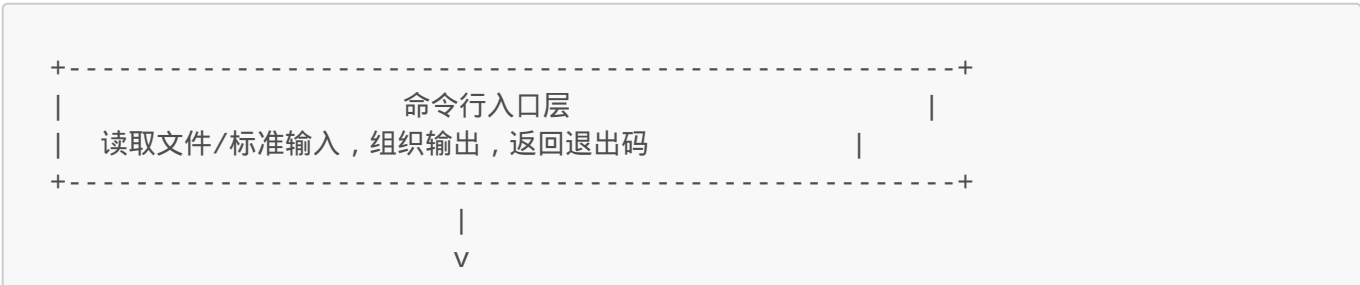
- 1. 出错位置的行号与列号；
- 2. 期望出现的结构；
- 3. 实际遇到的 token。

例如：`syntax error at 3:5: expected ';' after return, found '}'`

5. 系统总体设计

5.1 系统结构

系统整体采用分层模块化设计，如下所示：





5.2 模块划分

系统可划分为以下模块：

- 1. 输入模块
- 2. 词法分析模块
- 3. 语法分析模块
- 4. 输出模块
- 5. 测试模块

各模块职责如下。

5.2.1 输入模块

负责从两种来源读取源码：

- 1. 命令行指定的源文件；
- 2. 标准输入。

该模块统一返回字符串形式的源码，供词法分析器处理。

5.2.2 词法分析模块

负责按字符扫描源码，并输出 token 列表及词法错误列表。该模块位于 `My_Lexer` 工程中，是整个系统的前端基础。

5.2.3 语法分析模块

负责读取 token 列表，按照类 Rust 文法进行结构匹配，并输出分析成功或失败信息。该模块位于 `My_Parser` 工程中，复用了 `My_Lexer` 的 token 类型与位置信息。

5.2.4 输出模块

负责以可读格式打印结果：

1. 每个 token 的位置、种类与原始词素；
2. 词法错误信息；
3. 语法分析结果。

5.2.5 测试模块

负责对系统进行验证。测试采用以下两种方式：

1. 单元测试：验证单个分析函数的正确性；
2. 集成测试：对完整输入文件执行词法分析与语法分析，检查输出是否符合预期。

6. 系统详细设计

6.1 数据结构设计

6.1.1 位置信息 **Position**

为便于错误定位，系统为每个 token 和错误对象保存行号与列号：

```
Position {  
    line: usize,  
    column: usize,  
}
```

6.1.2 记号 **Token**

词法分析结果由 **Token** 结构表示，包含三部分：

1. **kind**：token 类型
2. **lexeme**：原始文本
3. **position**：起始位置

6.1.3 记号类型 **TokenKind**

使用 Rust 枚举表示不同种类 token，例如：

1. **Keyword(Keyword)**
2. **Identifier**
3. **Number**
4. **Assign**
5. **Plus**
6. **Arrow**
7. **LParen**
8. **RBrace**

这种设计能够充分利用 Rust 枚举的类型安全优势，使模式匹配更加清晰。

6.1.4 语法错误 **ParseError**

语法错误对象包含：

1. 错误信息
2. 错误位置

这使得程序能够输出精确的出错位置与原因。

6.2 词法分析模块详细设计

词法分析模块核心由一个扫描器对象维护以下运行状态：

1. 输入字符串
2. 当前扫描下标
3. 当前行号
4. 当前列号
5. token 列表
6. 错误列表

其主要处理步骤如下：

1. 调用 `skip_ignored()` 跳过空白和注释；
2. 根据当前字符分派到不同识别逻辑；
3. 若是字母或 `_`，进入标识符识别逻辑；
4. 若是数字，进入整数识别逻辑；
5. 若可构成双字符记号，则优先识别双字符记号；
6. 若是单字符界符或运算符，则直接输出；
7. 否则记录词法错误并继续扫描。

该设计具备以下优点：

1. 结构清晰，易于扩展新 token。
2. 统一保存位置信息，错误提示准确。
3. 通过最长匹配策略正确处理复合运算符。

6.3 语法分析模块详细设计

语法分析模块采用递归下降方式实现，核心结构为 `Parser`。其中主要成员为：

1. `tokens`：输入 token 列表
2. `current`：当前读取位置

6.3.1 程序分析

`parse_program()` 是入口函数。其逻辑为：

1. 持续读取函数声明；
2. 当读到 `#` 或 token 流末尾时结束；
3. 若 `#` 后仍有多余 token，则报错。

6.3.2 函数声明分析

`parse_function_decl()` 按以下顺序分析：

1. 匹配 `fn`
2. 匹配函数名
3. 匹配 `(`
4. 分析形参列表
5. 匹配 `)`
6. 若存在 `->`，分析返回类型
7. 分析函数体

这一逻辑与 PPT 中 1.1、1.4、1.5 的规则直接对应。

6.3.3 类型分析

`parse_type()` 支持以下类型结构：

1. 基本类型 `i32`
2. 引用类型 `&T`
3. 可变引用 `&mut T`
4. 数组类型 `[T;N]`
5. 元组类型 `(T1, T2, ...)`

虽然数组与元组并非最低强制要求，但实现后有助于后续扩展。

6.3.4 语句分析

`parse_statement()` 负责识别：

1. 空语句；
2. 变量声明语句
3. 返回语句
4. `while` 语句
5. `for` 语句
6. `break` 语句
7. `continue` 语句

对于赋值语句与表达式语句，系统在块分析函数中结合表达式分析进行判断。其原因是：

1. 赋值语句以表达式为前缀；
2. 需要先判断左侧表达式是否可赋值；
3. 再根据是否出现 `=` 与 `;` 来区分赋值语句、表达式语句和块尾表达式。

6.3.5 表达式分析

表达式采用分层下降法处理优先级：

1. `parse_expression()`：比较表达式
2. `parse_additive()`：加减表达式
3. `parse_multiplicative()`：乘除表达式
4. `parse_unary()`：一元表达式

5. `parse_postfix()`：函数调用、数组索引、元组字段
6. `parse_primary()`：数字、标识符、括号表达式、块表达式、数组、元组、`if` 表达式、`loop` 表达式

这种分层结构避免了直接使用左递归文法带来的问题，使程序能够自然地表达运算符优先级与左结合性。

6.3.6 左值判定

为了支持赋值语句，系统提供了左值判定逻辑。合法赋值左侧包括：

1. 标识符
2. 解引用表达式，如 `*a`
3. 数组下标表达式，如 `a[0]`
4. 元组字段表达式，如 `a.0`

若赋值左侧为普通常量或不可写表达式，则报告错误。例如 `1=2;` 会被识别为非法赋值。

6.4 命令行入口设计

命令行入口程序完成以下工作：

1. 读取文件或标准输入；
2. 调用 `My_Lexer::lex()` 获取词法分析结果；
3. 输出 token 序列；
4. 若存在词法错误，则直接输出错误并结束；
5. 否则调用 `parse_tokens()` 完成语法分析；
6. 输出 `syntax analysis succeeded` 或语法错误信息。

该设计的优点是流程清晰、层次分明，并且能够确保语法分析只在词法正确时进行。

7. 关键算法说明

7.1 最长匹配原则

词法分析中，多个 token 可能共享前缀。例如：

1. `=` 与 `==`
2. `>` 与 `>=`
3. `.` 与 `..`

系统采用最长匹配原则，即优先尝试双字符运算符。当双字符匹配失败时，再退化为单字符运算符。这样可避免将 `>=` 错误切分为 `>` 和 `=`。

7.2 递归下降与文法改写

递归下降分析器不能直接处理左递归文法，因此表达式部分不能直接照搬 PPT 中形如：

```
<加减表达式> -> <加减表达式> <加减运算符> <项>
```

这类左递归形式。为适应递归下降实现，需要在程序层面对其改写为“先读一个低层表达式，再循环读后续运算符和右操作数”的形式。其本质等价于：

```
Additive -> Multiplicative { ('+'|'-') Multiplicative }
```

这种改写不会改变可识别语言，但能够避免无限递归。

7.3 错误定位

无论在词法阶段还是语法阶段，系统都保存每个 token 的行列坐标。这样在出错时，能够直接报告：

1. 错误发生在哪一行、哪一列；
2. 当前读到的 token 是什么；
3. 理论上期望出现什么结构。

这种错误报告方式对调试实验程序非常重要。

8. 测试设计与结果

8.1 测试策略

系统采用“单元测试 + 集成测试”的方式验证正确性。

8.1.1 单元测试

单元测试主要覆盖：

1. 关键字后缀保持为标识符，如 `if123`
2. 保留字、运算符、数字的正确切分，如 `if=123`
3. 注释跳过
4. 复合运算符最长匹配
5. 基础文法样例
6. 扩展文法样例
7. 非法赋值左值
8. `#` 结束符处理

8.1.2 集成测试

集成测试复用了 `My_Lexer/tests/data/input.rs` 作为固定输入文件，并在 `My_Parser` 中增加了文件级测试：

1. 调用 `parser` 可执行程序读取该输入文件；
2. 将实际输出写入 `tests/data/output.txt`；
3. 校验输出是否等于词法分析结果加上 `syntax analysis succeeded`。

8.2 测试结果分析

测试结果表明：

1. 词法分析器能够正确识别保留字、标识符、常数、运算符和注释；
2. 语法分析器能够正确识别函数、变量声明、赋值、表达式、条件语句和循环语句；
3. 系统能够对非法赋值等错误情况进行准确定位与报告；

4. 当前实现已经满足 PPT 的最低强制要求，并支持部分扩展规则。

9. 系统特点与不足

9.1 系统特点

1. 采用模块化设计，词法与语法层职责清晰。
2. 词法分析支持最长匹配和错误恢复。
3. 语法分析采用递归下降，代码结构与文法结构对应紧密。
4. 输出信息包含行列位置，便于定位错误。
5. 测试覆盖了固定样例与关键边界情况。

9.2 当前不足

本系统目前主要完成“词法正确性”和“语法结构正确性”的检查，尚未深入到更完整的语义分析阶段。根据 PPT 最后一页的提示，以下功能仍可进一步扩展：

1. 赋值两侧的类型一致性检查；
2. 不可变变量的赋值检查；
3. `for` 循环可迭代对象的语义检查；
4. 更完整的数组、元组、引用与函数返回类型匹配检查；
5. 更完善的错误恢复机制与多错误收集能力。

10. 结论

本实验完成了一个面向类 Rust 子集的词法分析器和语法分析器。系统能够对输入源码进行扫描、生成 token 序列，并在此基础上依据给定文法检查程序结构的合法性。实验表明：

1. 通过手写扫描器可以较好地实现保留字识别、最长匹配和注释跳过等词法功能；
2. 通过递归下降方法可以清晰实现函数、语句、表达式等语法规则；
3. 将词法分析器和语法分析器解耦，有利于测试、维护和后续扩展；
4. Rust 的枚举、模式匹配与错误处理机制适合实现这类编译前端工具。

总体而言，本系统达到了课程设计对类 Rust 词法分析和语法分析的基本要求，也为后续继续实现语义分析、中间代码生成和优化提供了良好的基础。

11. 参考文献

以下文献与网络资源可直接列入报告，访问日期统一写为 2026 年 4 月 6 日。

11.1 图书

1. Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Pearson, 2011.
链接：<https://www.pearson.com/store/p/compilers-principles-techniques-and-tools/P200000003472>
2. Keith D. Cooper, Linda Torczon. *Engineering a Compiler (3rd Edition)*. Elsevier, 2022.
链接：<https://shop.elsevier.com/books/engineering-a-compiler/cooper/978-0-12-815412-0>

11.2 论文

1. Donald E. Knuth. "On the translation of languages from left to right." *Information and Control*, 8(6), 1965.
链接：<https://researchr.org/publication/knuth1965translation>
2. Ken Thompson. "Programming Techniques: Regular expression search algorithm." *Communications of the ACM*, 11(6), 1968.
链接：<https://cacm.acm.org/research/programming-techniques-regular-expression-search-algorithm/>
3. Donald E. Knuth. "Semantics of context-free languages." *Mathematical Systems Theory*, 2, 1968.
链接：<https://www.semanticscholar.org/paper/Semantics-of-context-free-languages-Knuth/0b61a17906637ece5a9c5e7e3e6de93378209706>

11.3 网络资源

1. The Rust Reference. *Tokens*. Rust 官方参考文档。
链接：<https://doc.rust-lang.org/reference/tokens.html>
2. The Rust Reference. *Comments*. Rust 官方参考文档。
链接：<https://doc.rust-lang.org/stable/reference/comments.html>
3. The Rust Reference. *Functions*. Rust 官方参考文档。
链接：<https://doc.rust-lang.org/reference/items/functions.html>
4. The Rust Reference. *if expressions*. Rust 官方参考文档。
链接：<https://doc.rust-lang.org/reference/expressions/if-expr.html>
5. The Rust Reference. *Loops and other breakable expressions*. Rust 官方参考文档。
链接：<https://doc.rust-lang.org/reference/expressions/loop-expr.html>
6. The Rust Reference. *Tuple and tuple indexing expressions*. Rust 官方参考文档。
链接：<https://doc.rust-lang.org/reference/expressions/tuple-expr.html>
7. The Rust Reference. *Array and array index expressions*. Rust 官方参考文档。
链接：<https://doc.rust-lang.org/beta/reference/expressions/array-expr.html>
8. The Rust Programming Language. *Functions*. Rust 官方教程。
链接：<https://doc.rust-lang.org/stable/book/ch03-03-how-functions-work.html>
9. The Rust Programming Language. *Control Flow*. Rust 官方教程。
链接：<https://doc.rust-lang.org/beta/book/ch03-05-control-flow.html>